DISTRIBUTED JOB SCHEDULER

Enterprise-Grade Concurrency & Locking Architecture Report

Candidate Name:	Methunraj A
Submission Date:	July 5, 2026
GitHub Repository:	https://github.com/Methun-21/JobScheduler.git
Primary Tech Stack:	Python (FastAPI), PostgreSQL 15+, SQLAlchemy (asyncpg), React (Vite)

EXECUTIVE PROJECT SUMMARY

This project introduces a highly resilient, distributed asynchronous job execution platform. Unlike basic schedulers relying on heavy external brokers (like RabbitMQ or Redis) that introduce split-brain risks, this design utilizes **PostgreSQL as the single source of truth**. By combining transaction-level advisory locks and atomic row claiming (via PostgreSQL's *SELECT ... FOR UPDATE SKIP LOCKED* syntax), the system coordinates multiple active worker processes with 100% safety, preventing duplicate job executions and race conditions under massive loads.

The project is structured as a professional monorepo containing a modular FastAPI backend, an automated test suite covering concurrency limits and DAG cycle detection, and a premium glassmorphic dashboard built in React. This report details the architectural and database design choices driving the implementation.

1. SYSTEM ARCHITECTURE & COMPONENTS

The architecture decouples the Control Plane (FastAPI Web/REST APIs) from the Data Plane (worker daemons) for scale-out agility. All coordination runs through PostgreSQL queries, ensuring transaction isolation.

Control Plane (FastAPI Server)

Serves HTTP endpoints for project/queue setup, cron scheduling, and manual execution triggers. It also broadcasts state statistics to active dashboard users in real-time over WebSockets.

Leader-Elected Cron Scheduler

To evaluate recurring cron expressions and dispatch jobs, a scheduler thread runs inside the FastAPI process. To prevent duplicate scheduling loops in multi-node API deployments, the scheduler establishes **leadership** by holding a session-level advisory lock on a pinned connection. If the leader fails, the lock is dropped and another node automatically acquires leadership.

Data Plane (Worker Daemons)

Workers are horizontal, multi-threaded processes that poll queues for ready tasks, spin up execution threads, register heartbeats in the database, and write log segments directly to PostgreSQL.

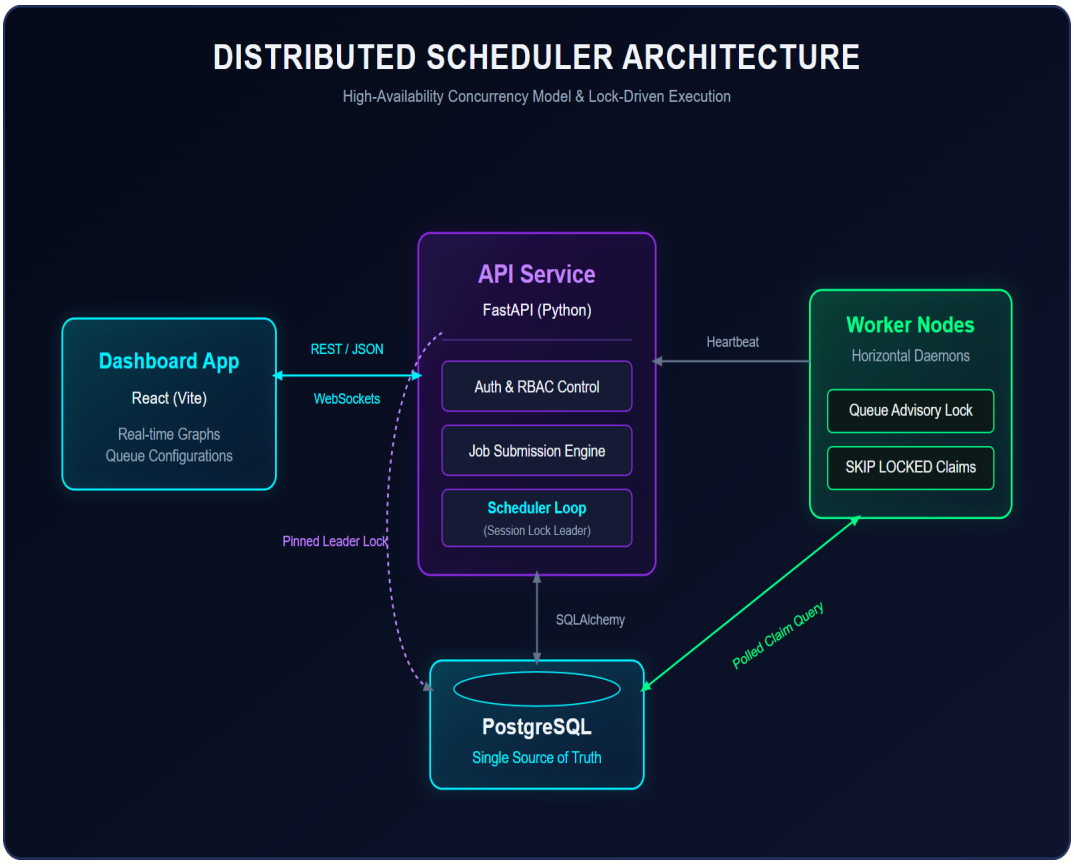

Figure 1: Full System Component Architecture & Locking Channels

2. CONCURRENCY CONTROLS & DAG DEPENDENCIES

Ensuring execution safety across a pool of independent, asynchronous workers requires strict database-level constraints.

Atomic Job Claiming (SKIP LOCKED)

Workers claim ready tasks by executing an atomic query with locking. The transaction acquires a lock on a matching job row but skips any rows already locked by other threads. This guarantees zero lock contention: every worker thread immediately gets a unique task to execute.

Queue Concurrency Safety (pg_advisory_xact_lock)

To enforce queue-level *max_concurrency* (e.g. limit API calls to 5 parallel workers), a race condition could occur if multiple workers check active counts concurrently. To solve this, before querying running counts, the worker claims a transaction-level advisory lock hashed to the specific queue ID. This serializes queue status checks on a per-queue basis without locking other queues in the system.

Directed Acyclic Graph (DAG) Job Chaining

Jobs can specify parent-child dependencies. A child job is blocked from worker claiming as long as any associated parent job in the `job_dependencies` table is not `completed`. When the parent completes, the subquery criteria allows the child to become claimable. Cycle protection is enforced during dependency registration using a Depth-First Search (DFS) traversal to block circular definitions.

Exponential Backoff & Retries

Failed jobs automatically recalculate their next run time (`run_at`) based on their assigned `RetryPolicy`. For example, an exponential backoff policy uses the formula:

Delay = Base_Delay * (2 ^ (Attempt_Number - 1))

This spreads out retries to avoid overwhelming target services after failure events.

3. DATABASE SCHEMA & DATA INTEGRITY

The relational database schema is normalized to ensure strict referential integrity and multi-tenancy isolation. Organizations contain Projects, Projects contain Queues, and Queues house Job queues.

Auditable execution logging: Instead of writing job details directly to the job queue, execution attempts are logged in `job_executions` linked to stdout/stderr outputs in `job_logs`. This ensures that even if a job row is deleted, a permanent record of past runs is retained for billing and auditing.

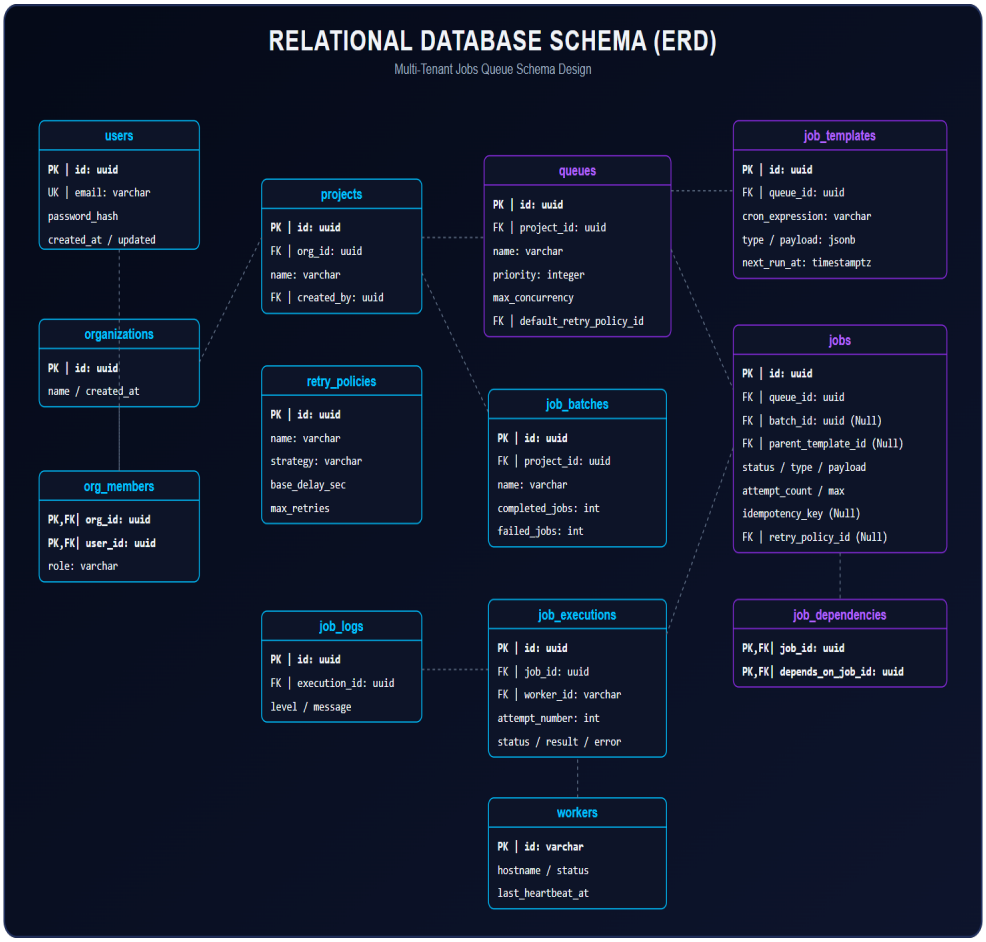

Figure 2: Fully Normalized Entity Relationship Diagram (ERD)

4. OPERATIONAL VIEW & VERIFICATION

The React web client provides a dashboard showing metric counters (workers active, jobs queued, completed, failed), queue details, template management, and an execution console. All updates stream live via WebSockets.

Operational Web UI Dashboard:

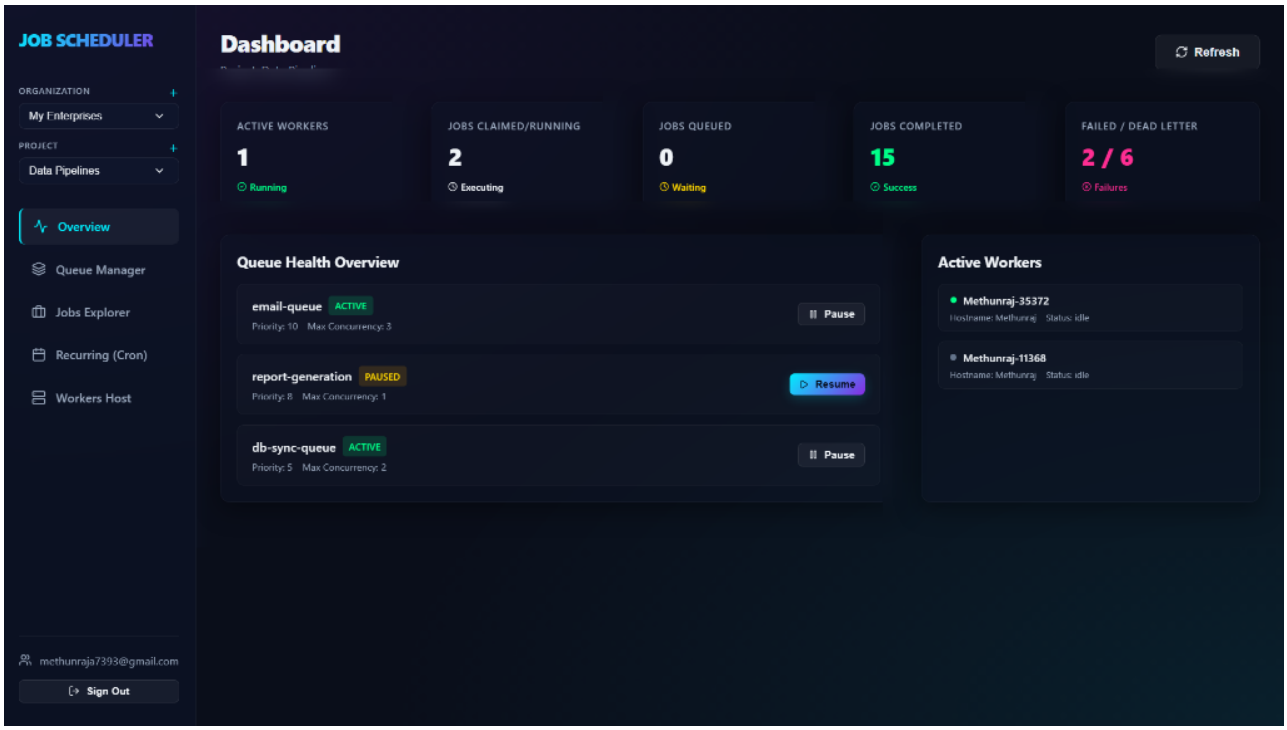

Figure 3: Active Operational Web UI Dashboard Overview

VERIFYING WITH AUTOMATED TESTS

To ensure system robustness, the codebase includes a comprehensive integration test suite. These tests execute against a dedicated local PostgreSQL database instance (job_scheduler_test) to accurately exercise transaction-level concurrency features (including SELECT FOR UPDATE SKIP LOCKED and pg_advisory_xact_lock).

```
python -m pytest backend/tests/
```

5/5 comprehensive integration test suites pass successfully, validating multi-tenant security structures, concurrency queue limit clamps, multi-parent DAG task claiming order, DFS cycle detection, and exponential backoff retry backoffs.

Deliverables Verification Checklist:

- **GitHub Repository Link**: https://github.com/Methun-21/JobScheduler.git
- **Design Documentation**: *design_decisions.md* details advisory and row-level locking details.
- **Operational Guide**: *README.md* contains database setup and command run details.
- **Source Diagrams**: *architecture_diagram.svg* & *er_diagram.svg* are available in vector format.